

THEORY OF COMPUTATION (CS302)

Equivalence of PDA and CFG

Learning Objectives

Understand the relationship between Pushdown Automata (PDA) and Context-Free Grammars (CFG), and learn conversion techniques between them.

Introduction

Context-Free Grammars and Pushdown Automata are two equivalent models used to describe context-free languages. Every CFG has an equivalent PDA and every PDA has an equivalent CFG.

Context-Free Languages

A Context-Free Language (CFL) is a language that can be generated by a CFG and recognized by a PDA.

Equivalence Theorem

For every Context-Free Grammar, there exists a Pushdown Automaton that accepts the same language. Similarly, for every PDA, there exists an equivalent CFG generating the same language.

CFG to PDA Conversion

Steps:

1. Push the start symbol onto the stack.
2. Replace variables using grammar productions.
3. Match terminal symbols with input symbols.
4. Accept when the input is consumed and the stack is empty.

Example CFG

$S \rightarrow aSb$
 $S \rightarrow ab$

The PDA simulates these productions using stack operations while processing the input string.

PDA to CFG Conversion

A CFG can be constructed from a PDA by creating grammar variables that represent transitions between states and stack operations.

Conversion Concept

Variables are introduced to represent the behavior of the PDA between pairs of states while processing stack symbols.

Why Equivalence is Important

- Proves that CFGs and PDAs have the same computational power.
- Helps in parser design.
- Provides theoretical foundations for compiler construction.

- Supports language recognition and generation.

Applications

- Compiler Design
- Syntax Analysis
- Parsing Techniques
- Formal Language Theory
- Programming Language Design

Worked Example

Grammar:

$S \rightarrow aSb \mid ab$

The equivalent PDA pushes symbols corresponding to variables and processes matching terminal symbols until the stack becomes empty.

Exam Important Questions

1. State the Equivalence Theorem.
2. Explain CFG to PDA conversion.
3. Explain PDA to CFG conversion.
4. Why are CFG and PDA considered equivalent?
5. Discuss applications of the equivalence theorem.

Summary

CFGs and PDAs are equivalent computational models for context-free languages. Understanding their relationship is essential for compiler design, parsing, and formal language theory.